IMPACT REPORT

Baseline vs. Final System

Five-metric comparison against the Module 1 baseline, the improvement cycles, and the tool-evolution drill summary.

Vijayalakshmi Ramu | October 2026 | github.com/Viji-qmofi/first-microservices-Viji-R-agentic-harness-cicd

RESULTS IN SIX LINES

1. On the Module 1 task the baseline passed (3/4/3). The final system passed 1 of 3 runs with the clarified prompt and 0 of 4 with the original, with and without governance.
2. Review latency was about 2.5 minutes in one timed review, against a 2-minute estimate for the baseline: no evidence of change.
3. Cycle time rose from 2m 46s to 3m 22s–3m 48s, mostly from a larger test suite plus about 1 minute of governance overhead.
4. Cost per build check rose from $0.35 to $0.38–$0.81: about +9% from the workload and +32% from governance. Orchestrated code changes cost $0.24 to $1.42 per run (7 runs); the live demo in the video cost $1.76, under the $2.00 limit.
5. Converting one agentic step to deterministic code cut its latency about 264 times (14 s to 0.053 s) and its token cost from $0.173 to $0.
6. Governance blocked what it should: the Orchestrator's earlier write is now denied (HO-06), 4 of 4 red-team prompts held, and CI gates fired on a weakened gate (PR #9), a one-minute timeout (PR #23) and a lowered cost budget (PR #24).

Measured results are kept separate from projections. Every figure comes from a recorded run unless it is labeled an estimate.

Executive summary

This system matured across four modules from an ungoverned, purely agentic pipeline into one with server-enforced role boundaries, a two-layer evaluation harness, CI/CD gating, and at least one step formally converted from LLM judgment to deterministic code with measured results. This report separates what was actually **measured** from what is **projected** – every figure in the measured section comes from a recorded run, any estimate inside it is labeled as one, and no figure in the projected section is presented as something that already happened. The headline comparison re-ran the Module 1 baseline task on the final system and on two ungoverned controls: governance costs about 32% more per run and about 1 minute more, and it did not cause the warning-coverage failures seen on re-run (details below).

Measured impact

Five-metric comparison against the Module 1 baseline (`module4_doc/module1-comparison.md`, `docs/iteration-log.md`)

The Module 1 task was re-run on the final governed system and on two ungoverned controls (`CLAUDE.md` and `.mcp.json` removed), same model, fresh container each time.

Metric	Baseline (Run 001, July 2026)	Final system (October 2026)	Reading
Quality (rubric scores)	3 / 4 / 3, Scope Pass: passed	Original prompt: Warning Coverage 2 and 1, both failed. Clarified prompts: 2, 4, 2, one pass in three	Worse than baseline on this task, partly recovered by prompt changes. The same failures occur without governance.
Review latency	about 2 min (estimate)	about 2.5 min (one timed review, n=1)	No evidence of change. Proxy: a single timed review, with the answer key in hand.
Defect rate (proxy: share of runs failing the rubric)	1 of 4 Module 1 runs	Original prompt: 4 of 4 failed (2 final, 2 controls). Clarified prompts: 2 of 3 failed	Not yet reliable. Direct defect counts are not available for a report-writing task, so run failure is the proxy.
Cycle time	2m 46s	3m 22s to 3m 48s (clarified prompts). 6m 38s (original prompt, Final Run 2)	Slower. Most of the rise is Maven on a larger test suite (order-service tests 1 to 41), plus about 1 minute of governance overhead. Run-to-run variance is large.
Cost per build check	$0.3494	Controls about $0.38. Final Run 2 $0.50. Clarified-prompt runs $0.38 to $0.81	About +9% from the workload and +32% (about $0.12) from governance. API-equivalent estimates from a subscription session.

Cost of orchestrated code changes (a different workload). The cost row above is the Module 1 build-check task. Across the 7 recorded orchestrated runs in `.eval-artifacts/` that report a cost, a run costs $0.24 to $1.42, against a per-task limit of $2.00 (four of the seven were over $1). The live demonstration in the walkthrough video, an order-service change that passed, cost $1.76 per `/usage`, the highest observed and still under the limit. It was not saved as a transcript, so it is not counted in the range. Figures are API-equivalent estimates.

Improvement cycles on this task (`docs/iteration-log.md`, Runs 005 and 006 and the rerun)

- Cycle 1: the prompt was changed to name runtime WARN/ERROR lines. Warning Coverage went from 1 to 2, and a separate scope failure surfaced (the agent ran `rm` and misreported it).
- Cycle 2: JVM warnings were named and a /tmp-only rule was added. Run 006 passed at 3/4/3.
- Rerun: the identical prompt scored 2 and failed, because it falsely reported "JVM warnings: None". The change did not hold reliably. A fixed-rule scan of the logs is recorded as the candidate fix and is not built.

> **Limits:** small n (one run per version), Final Run 1's timing is unreliable (interrupted and resumed), the repository changed since July (so only the control-to-final comparison isolates governance), and review latency has one observation.

`orchestrator_test` deterministic conversion (ADR-001, module3_doc/calibration-log.md, 2026-09-25)

- Latency: ~14s/run (agentic baseline, averaged over 3 runs) → ~0.053s/run (deterministic, averaged over 3 runs) – a ~264x reduction.
- Token cost: ~$0.173/run → $0/run – complete elimination for this step.
- Predictability: deterministic output confirmed byte-identical across repeated runs via `diff`; the agentic baseline was only confirmed consistent at the conclusion level, a real, disclosed gap in that measurement's own rigor.
- Verified via a real 4-run end-to-end regression check after integration, not isolated measurement alone – no regression attributable to the conversion.

Governance enforcement, before and after a real fix (ADR-005, module3_doc/calibration-log.md)

- Before: the Orchestrator's `update_entry` call on project storage succeeded in a real run (H0-06), despite being explicitly denied by written policy – a real near-miss, not caught by any check that existed at the time.
- After: the identical call, re-attempted post-fix, returns `authorization_denied` – verified directly via a real MCP call, not a simulated test. A positive control (`implementer`'s legitimate `write_entry`) confirmed the fix denies exactly what it should and nothing more.

CI/CD, first real run (docs/ci-step-design.md)

- Two real, previously-invisible defects (a `python`/`python3` base-image mismatch across three jobs; a missing `pytest` dependency) were caught the first time the pipeline actually executed on GitHub's runners – both had passed every local spot-check beforehand. Actions had been silently disabled by default (a forked-repo default, not a bug) until explicitly enabled, meaning these defects had been invisible for as long as the pipeline existed unexercised.
- `pipeline-integrity`'s deliberate-weakening test (`continue-on-error: true` added to `policy-gate` on a throwaway branch) was caught by two independent mechanisms simultaneously, confirmed in real commit history, then cleanly reverted.

Holdout measurement baseline (module3_doc/calibration-log.md, Module 3 Lab)

a clean, full-system measurement pass across six locked tasks found only 1 of 6 passed without a real, distinct finding – scope drift (agentic and human-caused), a structural gap in audit-log role identification, and a session-interruption recovery that briefly fabricated events before self-correcting. This is offered as a measured snapshot of system maturity at that point, not a claim the system is now defect-free – later modules (4.1's `ALLOWLIST_PATH` bug, 4.2's CI defects) found further real issues after this baseline, which is itself evidence the measurement process keeps finding real things rather than converging on false confidence.

Red-team boundary testing (eval/red-team-results.md)

4 of 4 planned boundary-violation attempts were correctly blocked – including one case blocked by an agent's own reasoning declining to exploit a documented, technically-available bypass, not by a technical enforcement layer. One unplanned, organic finding: a subagent fabricated a specific, confident security claim (a false prompt-injection report), then correctly retracted it only when directly pressed to produce verbatim evidence – recorded as its own distinct failure category, not folded into the boundary-test results.

Projected impact (not measured; clearly distinguished from the above)

No live team's prior manual process exists for this project to measure a true before/after against (`module4_doc/workflow-scoping.md`). The following is **context**, not a claim about this project's own history:

- Published industry research on code review overhead (Google's internal engineering-productivity data, cited via secondary sources; SonarSource/Infragistics developer-time surveys) suggests the general category of work this system addresses typically costs several hours per developer per week in review/coordination overhead, separate from actual coding time.
- *If* the `orchestrator_test`-style conversion pattern (measure an agentic step, convert if it clears all four signals, verify with a real regression check) were applied to other qualifying steps in this system, similar latency/cost reductions could reasonably be expected for steps that share the same four-signal profile – this is a reasonable extrapolation from one real data point, not a second measurement. `ADR-002` (the line-ending-noise step) was evaluated against this same process and correctly found *not* to qualify for full conversion, which is itself evidence the process doesn't just rubber-stamp every candidate toward "yes."

Tool-evolution drill summary (drill type: convert another step)

The drill converted `orchestrator_test`, the step where the Orchestrator interprets raw `./mvnw test` output, from LLM judgment to a deterministic script (ADR-001). It is one of the four qualifying drill types named in the capstone rubric. Full write-up: `module4_doc/tool-evolution-drill.md`.

What was changed

`orchestrator_test` – the step where the Orchestrator interprets raw `./mvnw test` output to decide pass/fail – was converted from LLM narration to a deterministic script (`scripts/parse_test_result_deterministic.py`), per `ADR-001`. This is a real production change to the live orchestration pipeline, not a simulated or isolated exercise: `CLAUDE.md`'s own Orchestrator Instructions were edited to point at the script, and every orchestrated task run since has used the converted version.

What broke

Nothing in the converted step's own behavior regressed – confirmed across 4 real end-to-end orchestrated runs (2 holdout-task reruns, 2 genuine new-diff development tasks) after integration. What the regression-check *process* surfaced instead were two real defects in how evidence itself was being recorded, neither caused by the conversion but both found only because the harness was run against real post-integration evidence rather than assumed correct:

1. **A transcript recorded `cost_usd` as the string `"not_measured"` instead of JSON `null`.** Running `eval/test_deterministic.py` against it crashed with an unhandled `TypeError` (`'>' not supported between instances of 'str' and 'float'`) rather than a clean fail – the harness's own `check_cost` function had no defensive handling for a malformed field type. This is itself a known-gap finding worth carrying into future harness work, even though the fix applied here was correcting the transcript, not hardening the checker.
2. **A second transcript's `expected_path` listed `implementer` before `orchestrator_plan_review`** – the reverse of what actually happened and of the established review-before-implementation convention used everywhere else in this project. `check_role_order` correctly failed against this, exactly as designed.

> Neither defect was caught by manual read-through before the harness ran – both transcripts looked complete and plausible on inspection. The harness catching them is the real evidence this drill produced: **running the real evaluation harness against real new evidence surfaced two defects that manual review of the same evidence had already missed.**

What improved

Measured directly, not estimated (`module3_doc/calibration-log.md`, 2026-09-25 entries):

- **Latency:** ~264x reduction (14s/run average, agentic baseline → 0.053s/run average, deterministic).
- **Cost:** 100% token-cost elimination for this step ($0.173/run → $0/run).
- **Predictability:** the deterministic version produces byte-identical output across repeated runs on the same input, confirmed via `diff`, not merely a consistent conclusion – the agentic baseline was only confirmed consistent at the conclusion level, not the full-text level, a real, disclosed limitation of the baseline measurement itself.

How the eval harness caught regressions

`eval/test_deterministic.py` run against both new-diff transcripts caught the `expected_path` defect directly (`check_role_order` failing with a precise, correct error naming the actual vs. expected sequence). The `cost_usd` type defect was caught as an unhandled crash rather than a clean, named failure – a real, acknowledged limitation in the harness's own robustness, not a false negative, since the run genuinely could not be scored until the transcript was corrected. Both required a human to diagnose the root cause and apply the fix; the harness's role was surfacing that *something* was wrong, not diagnosing *why*.

Separately, two of the four regression-check runs (the holdout-task reruns) turned out to require no new implementation at all, since the underlying work already existed from an earlier bulk port. This was itself discovered via the same discipline this drill depends on throughout – `git diff -w` checked directly rather than trusting `planner`'s "already implemented" claim at face value (see `skills/verify-before-trusting`).

Final result

> `ADR-001`: **Accepted.** Full before/after measurement, integration, and a real 4-run end-to-end regression check are all recorded in `module3_doc/calibration-log.md`, with every number traceable to a specific, named entry rather than summarized from memory.